Object Storage Mode

Infinite Storage. Stateless Compute.

The Problem

Traditional database storage doesn't fit cloud-native reality.

Local Disk Is Finite

Scaling storage means bigger machines, costly migrations, and downtime.

Ephemeral Compute

Containers, Kubernetes pods, spot instances - local storage disappears when nodes die.

Replication Overhead

Traditional replication requires complex consensus protocols and dedicated infrastructure.

The Solution

Store SSTables in S3-compatible storage. Use local disk as a bounded cache.
Every compute node becomes stateless. Storage becomes infinite.

TidesDB Engine

Write path, MVCC, compaction

→

Local NVMe Cache

Hot data at disk speed

→

S3 / MinIO Object Store

Infinite durable storage

Compatible w/ AWS S3 · MinIO · Google Cloud Storage · Any S3 API

Four-Tier Storage Hierarchy

Data flows down through tiers as it ages and flows back up when accessed.

HOT	**In-Memory Block Cache** Microsecond reads	NUMA-aware CLOCK eviction cache
WARM	**Local Disk - Open Handles** µs to ms reads	LRU file handle cache (default 512)
COLD	**Local Disk - Closed Files** Reopened on access	Reopened on demand, then promoted
FROZEN	**Object Store (S3)** HTTP range requests	Point lookups via single HTTP range request

A node with 1TB NVMe can serve a 100TB dataset, hot data at local speed, cold data fetched on demand.

What this means for TideSQL?

Object Store Mode plugs directly into MariaDB via TideSQL.
Every SQL feature works identically - transactions, indexes, DDL, partitioning.

```
my.cnf

[mariadb]
tidesdb_object_store_backend = S3
tidesdb_s3_endpoint = s3.amazonaws.com
tidesdb_s3_bucket = my-tidesdb-bucket
tidesdb_s3_region = us-east-1
tidesdb_objstore_local_cache_max = 536870912
```

- All SQL works unchanged - CREATE TABLE, SELECT, INSERT, transactions
- Existing applications require zero code changes
- Each table's SSTables automatically tier to S3

Read Replicas & Automatic Failover

Horizontal read scaling with disposable, stateless compute nodes.

Read Replicas

- Point any MariaDB instance at the same S3 bucket
- Replicas poll MANIFEST for updates and replay WAL segments
- Each replica maintains its own local cache
- Need 50 read replicas? Spin up 50 pods
- Near-real-time reads with configurable sync interval

Primary Promotion

```
SET GLOBAL tidesdb_promote_primary = ON;
```

- Final MANIFEST sync and WAL replay
- Creates local WAL for crash recovery
- Starts accepting writes immediately
- Fast, metadata already in memory from sync loop
- K8s controller automates detection + promotion

Durability & RPO Guarantees

Configurable data loss window, from zero-loss to high-throughput batching.

Setting	RPO	Tradeoff
`wal_sync_on_commit = ON`	**Zero**	One HTTP round-trip per commit
`wal_sync_threshold = 1MB`	**~1MB of writes**	Periodic sync based on write volume
`wal_sync_threshold = 0`	One flush cycle	Maximum throughput, no periodic sync

On clean shutdown, there is zero data loss regardless of configuration.

Real-Time Monitoring

```
SHOW ENGINE TIDESDB STATUS      uploads, failures, queue depth, cache usage, replica mode
```

What This Enables

Cloud-Native SQL at Any Scale

Partition by time, let S3 hold everything, scale reads by adding stateless pods. 100TB+ with standard SQL.

Serverless & Ephemeral Compute

Containers and spot instances become disposable compute layers. If a node dies, another takes over from S3 in seconds.

Cost-Optimized Data Tiering

Hot data on NVMe, cold data on S3 at $0.023/GB. No manual lifecycle policies - the engine manages it automatically.

Global Multi-Region Reads

Read replicas in any region, each pointing to the same S3 bucket. Serve reads from the nearest location.

Disaster Recovery Built-In

S3 is the durable copy. Cold start recovery downloads MANIFEST and fetches data on demand. No backup scripts.

Kubernetes Deployment

Ship with included manifests - primary, replicas, and automated failover controller.

`primary.yaml`

StatefulSet for the primary MariaDB with S3 configuration. Single writer node.

`replica.yaml`

Deployment for read replicas. Scale horizontally with kubectl scale.

`failover-controller.yaml`

Checks primary every 5 seconds. After 3 failures, promotes a replica automatically.

Failover Timeline

`0s`
Primary goes down

→

`5s`
First health check fails

→

`15s`
Third consecutive failure

→

`~16s`
Replica promoted - writes resume

Bringing it all together

Infinite storage. Stateless compute. Standard SQL.
One engine. Any scale.

tidesdb.com

github.com/tidesdb